

Solutions for Homework Assignment #3

Answer to Question 1.

a. Suppose that f is computable and let M_f be a TM that computes f . We construct a 2-tape TM M that decides $\mathcal{G}(f)$ as follows:

on input $\langle x, f(x) \rangle$:

- 1 copy x (the first part of the input string) from tape 1 to tape 2
- 2 run M_f (on input x) using tape 2
- 3 compare the contents of tape 2 (when M_f halts) to the second part of the input on tape 1
- 4 **if** they are the same (i.e., tape 2 contains $f(x)$) **then** accept $\langle x, f(x) \rangle$
- 5 **else** reject $\langle x, f(x) \rangle$

Clearly M accepts $\langle x, y \rangle$ if $y = f(x)$ and rejects $\langle x, y \rangle$ if $y \neq f(x)$. So M decides $\mathcal{G}(f)$.

Conversely, suppose that $\mathcal{G}(f)$ is decidable and let M be a TM that decides $\mathcal{G}(f)$. We construct a 3-tape TM M_f that computes f .

on input x :

- 1 $y :=$ string on tape 2 (initially empty)
- 2 copy $\langle x, y \rangle$ on tape 3 (x is on tape 1 and y on tape 2)
- 3 run M (on input $\langle x, y \rangle$) on tape 3
- 4 **if** M accepts $\langle x, y \rangle$ **then** erase the input x from tape 1 and replace it by y (copying y from tape 3), and halt
- 5 **else** replace the string on tape 2 by the next string in shortlex order and go to step 2

To see why this computes $f(x)$ we note that, because M is a decider for $\mathcal{G}(f)$, step 3 will always terminate and M will accept $\langle x, y \rangle$ if and only if $y = f(x)$. Since f is a (total) function $f(x)$ is defined and so there is some binary string y such that $f(x) = y$. So, eventually, the string y on tape 2 will be $f(x)$, M will accept $\langle x, f(x) \rangle$ in step 3, and M_f will halt with the correct output, $f(x)$, on tape 1 in step 4.

b. Suppose that f is a partial computable function and let M_f be a TM that computes f . The same construction as in part (a) results in a TM that recognizes $\mathcal{G}(f)$. Notice that now in step 2 M_f runs forever on input x if f is not defined on x . In this case the constructed TM M will also run forever on any input $\langle x, - \rangle$ (no matter what the second component is). This is not a problem because now we only want a TM that recognizes $\mathcal{G}(f)$, not one that decides $\mathcal{G}(f)$.

For the converse, suppose that $\mathcal{G}(f)$ is recognizable and let M be a TM that recognizes $\mathcal{G}(f)$. The construction in part (a) does not work any more because in step 3 M is only a recognizer and might run forever on some input $\langle x, y \rangle$, even though there is some y that is after y' in shortlex order such that M accepts $\langle x, y \rangle$. In such a case we want M_f to output y , not to run forever. The solution, of course, is to generate pairs (y, i) in dovetail order, where y is a binary string and i is a positive integer, and change step 3 to run M on input $\langle x, y \rangle$ for i steps. This way, if there is a pair $\langle x, y \rangle$ such that M accepts $\langle x, y \rangle$ in some number of steps, say i , when we execute step 3 having reached (y, i) in dovetail order, we will run M on $\langle x, y \rangle$ for long enough to accept the pair, and our TM M_f will output y , as it should.

Answer to Question 2. We will use a 2-tape NTM. Tape 1 holds the input and tape 2 will hold the (presumed) repeated substring y that the NTM will “guess”. In more detail, the NTM works as follows:

(1) While in the initial state q_0 , scan the input left to right; in each move the TM makes a nondeterministic choice:

- either remain in state q_0 and continue scanning to the right,
- or enter a new state q_1 that marks the beginning of the (presumed) repeated substring y .

- (2) Once the TM has entered state q_1 , it makes 15 moves to the right; in each of these moves it copies the symbol it sees on tape 2, and moves the head of tape 2 to the right. So after these 15 moves, it has copied the (presumed) repeated substring y on tape 2. (For these moves the TM uses 15 new states to remember how many symbols it has copied.)
- (3) It then moves the head of tape 2 to the leftmost cell and enters a new state q_2 in which it continues the scan of the input.
- (4) While in state q_2 , the TM makes a nondeterministic choice:
 - either remain in state q_2 and continue scanning to the right,
 - or enter a new state q_3 that marks the beginning of the second occurrence of the (presumed) repeated substring y .
- (5) Once the TM has entered state q_3 , it compares the next 15 symbols on the input tape (tape 1) to the 15 symbols it has copied on tape 2, symbol by symbol, by moving one cell to the right in each move. If during these 15 moves it finds a mismatch, it rejects. If all 15 symbols match, it accepts.
- (6) If the TM reaches the leftmost blank on tape 1, it rejects.

If $x \in A$ then there is a sequence of nondeterministic choices (namely going from q_0 to q_1 when it moves the tape head over the beginning of the first occurrence of a repeated string y and going from q_2 to q_3 when it moves the tape head over the beginning of a subsequent occurrence of y) that result in the TM entering the accept state. If $x \notin A$ there is no such sequence of nondeterministic choices, and the TM rejects. So this NTM recognizes A .

Answer to Question 3.

a. This statement is false.

Counterexample: Let $A = \text{UNIV}$ and $B = \overline{\text{UNIV}}$. Then $A \cup B = \Sigma^*$ and $A \cap B = \emptyset$, which are both decidable. Yet both A and B are undecidable.

b. This statement is false.

Counterexample: Let $A = \{\langle M, x \rangle : M \text{ halts on } x \text{ in at most 100 steps}\}$ and $B = \overline{\text{UNIV}}$. Note that $A \cap B = \{\langle M, x \rangle : M \text{ rejects } x \text{ in at most 100 steps}\}$. A is decidable by running the universal Turing machine on input $\langle M, x \rangle$ for at most 100 steps. $A \cap B$ is also decidable for essentially the same reason (we now check that M rejects x , rather than that it halts on x , within 100 steps), and so *a fortiori* it is recognizable. However, B is not recognizable.

c. This statement is true.

Proof: Let M_A and M_B be recognizers for A and B , respectively; $M_{\cup}$ be a decider for $A \cup B$, and $M_{\cap}$ be a decider for $A \cap B$. Shown below is a decider D_A for A . A decider for B can be obtained similarly (be exchanging the roles of A and B).

$D_A :=$ on input x :

```

1   run  $M_{\cup}$  on  $x$ 
2   if  $M_{\cup}$  rejects then reject
3   else
4     run  $M_{\cap}$  on  $x$ 
5     if  $M_{\cap}$  accepts then accept
6     else
7       while true do
8         run  $M_A$  on  $x$  for one more step
9         if  $M_A$  accepts then accept
10        else
11          run  $M_B$  on  $x$  for one more step
12          if  $M_B$  accepts then reject

```

Suppose $x \in A$. Then either $x \in A \cap B$, in which case D_A accepts x in line 5; or $x \in A - B$, in which case $M_{\cap}$ rejects x and M_B does not accept x after any number of steps, so D_A accepts x in line 9 of some iteration of the while loop.

Conversely suppose $x \notin A$. Then either $x \notin B$, in which case D_A rejects x in line 2; or $x \in B - A$, in which case M_B accepts x after some number of steps and M_A does not accept x after any number of steps, so D_A rejects x in line 12 of some iteration of the while loop.

Do not post on the internet without copyright owner's written permission